

MI WLAN API

Version 2.10

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
2.03	Initial release	08/14/2019
2.10	Update api	11/01/2021

TABLE OF CONTENTS

REVISION HISTORY i

TABLE OF CONTENTS ii

1. API REFERENCE 1

- 1.1. Description 1
- 1.2. API List 1
 - 1.2.1 MI_WLAN_Init 1
 - 1.2.2 MI_WLAN_DeInit 2
 - 1.2.3 MI_WLAN_Open 3
 - 1.2.4 MI_WLAN_Close 3
 - 1.2.5 MI_WLAN_Connect 4
 - 1.2.6 MI_WLAN_Disconnect 5
 - 1.2.7 MI_WLAN_Scan 5
 - 1.2.8 MI_WLAN_GetStatus 6
 - 1.2.9 MI_WLAN_GetWlanChipVersion 7

2. DATA TYPE 8

- 2.1. Module Related Data Type 8
 - 2.1.1 WLAN_HANDLE 8
 - 2.1.2 MI_WLAN_Security_e 9
 - 2.1.3 MI_WLAN_Encrypt_e 9
 - 2.1.4 MI_WLAN_NetworkType_e 10
 - 2.1.5 MI_WLAN_Authentication_Suite_e 10
 - 2.1.6 MI_WLAN_WPAStatus_e 11
 - 2.1.7 MI_WLAN_InitParams_t 11
 - 2.1.8 MI_WLAN_OpenParams_t 12
 - 2.1.9 MI_WLAN_ConnectParam_t 12
 - 2.1.10 MI_WLAN_Quality_t 13
 - 2.1.11 MI_WLAN_Cipher_t 14
 - 2.1.12 MI_WLAN_APIInfo_t 14
 - 2.1.13 MI_WLAN_ScanParam_t 15
 - 2.1.14 MI_WLAN_ScanResult_t 16
 - 2.1.15 MI_WLAN_Status_sta_t 16
 - 2.1.16 MI_WLAN_Status_host_t 17
 - 2.1.17 MI_WLAN_Status_ap_t 18

3. ERROR CODE 19

4. APPENDIX A 20

- 4.1. wlan.json 20

1. API REFERENCE

1.1. Description

This module (wireless local area network) provides simple scanning of wifi signals. The connection function provides support for AP and STA modes.

1.2. API List

Name of API	Function
MI_WLAN_Init	Initialize WLAN device
MI_WLAN_DeInit	Deinitialize WLAN device
MI_WLAN_Open	Enable WLAN device and set parameters
MI_WLAN_Close	Close WLAN device
MI_WLAN_Connect	Connect Wi-Fi hotspot or enable AP mode
MI_WLAN_Disconnect	Disconnect Wi-Fi hotspot or disable AP mode
MI_WLAN_Scan	Scan available hotspots
MI_WLAN_GetStatus	Get the connection status
MI_WLAN_GetWlanChipVersion	Get the WLAN device FW version

1.2.1 MI_WLAN_Init

➤ Function

Initialize WLAN device

➤ Syntax

```
MI_RESULT MI_WLAN_Init(MI\_WLAN\_InitParams\_t *pstInitParams);
```

➤ Parameter

Parameter Name	Description	Input/Output
pstInitParams	Pointer to WLAN device initialization parameter	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h

- Library: libmi_wlan libcjson libcrypto libssl libnl
- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

- The initialization of WLAN device relies on a json configuration file called wlan.json which located at /config/wifi/wlan.json or /customer/wifi/wlan.json.
- This parameter does not accept Null value currently.
- Since the WLAN module contains many configurable items and the data types are both complicated and indefinite, a json configuration file is introduced for quicker understanding. For details on the configuration file and the configurable items, please refer to [APPENDIX A](#).

➤ Example

The example code below realizes the setup of WLAN device initialization:
 MI_WLAN_InitParams_t stParm = {"/config/Wi-Fi/wlan.json"};
 MI_WLAN_Init(&stParm);

1.2.2 MI_WLAN_DeInit

➤ Function

Deinitialize WLAN device

➤ Syntax

MI_RESULT MI_WLAN_DeInit(void);

➤ Parameter

N/A

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h
- Library: libmi_wlan libcjson libcrypto libssl libnl
- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

N/A

➤ Example

```
MI_RESULT ret;
ret = MI_WLAN_DeInit();
if(MI_SUCCESS != ret)
{
    return ret;
}
```

1.2.3 MI_WLAN_Open

➤ Function

Enable WLAN device and set parameters

➤ Syntax

```
MI_RESULT MI_WLAN_Open(MI\_WLAN\_OpenParams\_t *pstParam);
```

➤ Parameter

Parameter Name	Description	Input/Output
pstParam	Pointer to WLAN device parameter	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h
- Library: libmi_wlan libcjson libcrypto libssl libnl
- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

To call this function successfully, the WLAN device must be initialized already.

➤ Example

Please refer to the example illustrated in [MI_WLAN_OpenParams_t](#).

1.2.4 MI_WLAN_Close

➤ Function

Close WLAN device

➤ Syntax

```
MI_RESULT MI_WLAN_Close(MI\_WLAN\_OpenParams\_t *pstParam);
```

➤ Parameter

Parameter Name	Description	Input/Output
pstParam	Pointer to WLAN device parameter	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h
- Library: libmi_wlan libcjson libcrypto libssl libnl

- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

To call this function successfully, the WLAN device must be initialized already.

➤ Example

```
MI_RESULT ret;
MI_WLAN_Open_Params_t g_stOpenParam={E_MI_WLAN_ENTWORKTYPE_INFRA};
ret = MI_WLAN_Close (&g_stOpenParam);
if(MI_SUCCESS != ret)
{
    printf(" wlan close fail\n");
    return ret;
}
```

1.2.5 MI_WLAN_Connect

➤ Function

Connect Wi-Fi service

➤ Syntax

```
MI_RESULT MI_WLAN_Connect(WLAN_HANDLE *hWlan, MI\_WLAN\_ConnectParam\_t
*pstConnectParam);
```

➤ Parameter

Parameter Name	Description	Input/Output
hWlan	WLAN handle pointer 1. (<0) indicates a new sta connection and a sensible ID will be assigned if the connection is set up. (sta mode) 2. (>0) indicates a connection to this very SSID was built before. 3. (==WLAN_HANDLE_AP) indicates ap mode.	Input/Output
pstConnectParam	Pointer to Wi-Fi connection parameter setting See MI_WLAN_ConnectParam_t .	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h
- Library: libmi_wlan libcjson libcrypto libssl libnl
- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

- If the Wi-Fi hotspot does not have a password, you need to input the empty string "".
- Currently the timeout parameter does not work. The connection timeout is specified by

the parameter connect:infra:dhcp -T -t in wlan.json. See [APPENDIX A](#).

➤ Example

```
WLAN_HANDLE wlanHdl = -1; //std mode
MI_WLAN_ConnectParam_t stConnectParam = \
    {E_MI_WLAN_SECURITY_WPA2, "123456", "", 5000};
MI_WLAN_Connect(&wlanHdl, &stConnectParam);
```

1.2.6 MI_WLAN_Disconnect

➤ Function

Disconnect Wi-Fi service

➤ Syntax

```
MI_RESULT MI_WLAN_Disconnect(WLAN_HANDLE hWlan);
```

➤ Parameter

Parameter Name	Description	Input/Output
hWlan	WLAN handle	Input

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h
- Library: libmi_wlan libcjson libcrypto libssl libnl
- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

Make sure hWlan > 0

➤ Example

```
WLAN_HANDLE wlanHdl = -1;
MI_WLAN_ConnectParam_t stConnectParam = \
    {E_MI_WLAN_SECURITY_WPA2, "123456", "", 5000};
MI_WLAN_Connect(&wlanHdl, &stConnectParam);
MI_WLAN_Disconnect(wlanHdl);
```

1.2.7 MI_WLAN_Scan

➤ Function

Scan currently available Wi-Fi connections

➤ Syntax

```
MI_RESULT MI_WLAN_Scan(MI\_WLAN\_ScanParam\_t *pstParam, MI\_WLAN\_ScanResult\_t
    *pstResult);
```

➤ Parameter

Parameter Name	Description	Input/Output
pstParam	Scan parameter pointer, currently not in use	Input
pstResult	Pointer to information on scanned available Wi-Fi hotspots	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h
- Library: libmi_wlan libcjson libcrypto libssl libnl
- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

After calling MI_WLAN_Scan in the parameter pstResult, u8APNumber defaults to the maximum value MI_WLAN_MAX_APINFO_NUM, indicating the number of hotspot information you need to obtain. After the scan is completed, it will be set to the actual number of hotspots.

➤ Example

```
MI_WLAN_ScanResult_t stRst;
while(1)
{
    MI_WLAN_Scan(NULL, &stRst); /*stRst.u8APNumber= MI_WLAN_MAX_APINFO_NUM by
    default.*/
    for(i = 0 ; i < stRst.u8APNumber; i ++ )
    {
        printf("%s %d %s\n", __FUNCTION__, __LINE__, stRst.stAPIInfo[i].au8SSID);
    }
    sleep(3);
}
```

1.2.8 MI_WLAN_GetStatus

➤ Function

Get all currently available Wi-Fi connection status.

➤ Syntax

```
MI_RESULT MI_WLAN_GetStatus(WLAN_HANDLE hWlan, MI\_WLAN\_Status\_t *pstStatus);
```

➤ Parameter

Parameter Name	Description	Input/Output
hWlan	Wlan handle	Input
pstStatus	All currently available Wi-Fi connection status	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h
- Library: libmi_wlan libcjson libcrypto libssl libnl
- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

MI_WLAN_Status_t is a union, which returns, according to the network type configured when the connection is enabled, information on the connected hotspots or the host connected to the current device.

➤ Example

N/A

1.2.9 MI_WLAN_GetWlanChipVersion

➤ Function

Get the WLAN device FW version.

➤ Syntax

```
MI_RESULT MI_WLAN_GetWlanChipVersion(MI_U8 *pstChipVersion);
```

➤ Parameter

Parameter Name	Description	Input/Output
pstChipVersion	WLAN device FW version	Output

➤ Return Value

- Zero: Successful
- Non-zero: Failed, see [error code for](#) details

➤ Dependency

- Header: mi_wlan.h mi_wlan_datatype.h
- Library: libmi_wlan libcjson libcrypto libssl libnl
- Executable File: iwlist wpa_cli wpa_supplicant

➤ Note

N/A

➤ Example

N/A

2. DATA TYPE

2.1. Module Related Data Type

Data Type	Definition
WLAN_HANDLE	Define the WLAN handle
MI_WLAN_Security_e	Define the WLAN security type
MI_WLAN_Encrypt_e	Define the WLAN encryption type
MI_WLAN_NetworkType_e	Define the WLAN device network type
MI_WLAN_Authentication_Suite_e	Define the WLAN authentication suite
MI_WLAN_WPAStatus_e	Define the WLAN connection status
MI_WLAN_InitParams_t	Define the WLAN device initialization parameter
MI_WLAN_OpenParams_t	Define the WLAN device enable parameter
MI_WLAN_ConnectParam_t	Define the WLAN connection parameter
MI_WLAN_Quality_t	Define the Wi-Fi hotspot signal quality
MI_WLAN_Cipher_t	Define the Wi-Fi hotspot cipher information
MI_WLAN_APIInfo_t	Define the Wi-Fi hotspot information
MI_WLAN_ScanParam_t	Define the Wi-Fi hotspot scan parameter information
MI_WLAN_ScanResult_t	Define the Wi-Fi hotspot scan result
MI_WLAN_Status_sta_t	Define the connected Wi-Fi hotspot information
MI_WLAN_Status_host_t	Define the connected host information
MI_WLAN_Status_ap_t	Define all connected host information
MI_WLAN_Status_t	Union of MI_WLAN_Status_host_t and MI_WLAN_Status_sta_t

2.1.1 WLAN_HANDLE

- Description
WLAN device handle, used to mark and distinguish Wi-Fi connection
- Definition
typedef MI_S32 WLAN_HANDLE
- Note
N/A
- Related Data Type and Interface
N/A

2.1.2 MI_WLAN_Security_e

- Description
Wi-Fi communication security type

- Definition


```
typedef enum
{
    /// WLAN module security key off
    E_MI_WLAN_SECURITY_NONE      = 1 << 0,
    /// WLAN module security key unknow
    E_MI_WLAN_SECURITY_UNKNOWTYPE = 1 << 1,
    /// WLAN module security key WEP
    E_MI_WLAN_SECURITY_WEP      = 1 << 2,
    /// WLAN module security key WPA
    E_MI_WLAN_SECURITY_WPA      = 1 << 3,
    /// WLAN module security key WPA2
    E_MI_WLAN_SECURITY_WPA2     = 1 << 4,
    /// WLAN module max
    E_MI_WLAN_SECURITY_MAX      = 0xff,
} MI_WLAN_Security_e;
```

- Member

Member	Description
E_MI_WLAN_SECURITY_NONE	WLAN module security key off
E_MI_WLAN_SECURITY_UNKNOWTYPE	WLAN module security key unknow
E_MI_WLAN_SECURITY_WEP	WLAN module security key WEP
E_MI_WLAN_SECURITY_WPA	WLAN module security key WPA
E_MI_WLAN_SECURITY_WPA2	WLAN module security key WPA2
E_MI_WLAN_SECURITY_MAX	WLAN module max

- Note
N/A
- Related Data Type and Interface
N/A

2.1.3 MI_WLAN_Encrypt_e

- Description
Wi-Fi encryption type

- Definition


```
typedef enum
{
    /// WLAN module encrypt type none
    E_MI_WLAN_ENCRYPT_NONE      = 1 << 0,
```

```

/// WLAN module encrypt type unknown
E_MI_WLAN_ENCRYPT_UNKNOWN    = 1 << 1,
/// WLAN module encrypt type WEP
E_MI_WLAN_ENCRYPT_WEP       = 1 << 2,
/// WLAN module encrypt type TKIP
E_MI_WLAN_ENCRYPT_TKIP      = 1 << 3,
/// WLAN module encrypt type AES
E_MI_WLAN_ENCRYPT_AES       = 1 << 4,
/// WLAN module max
E_MI_WLAN_ENCRYPT_MAX       = 0xff,
} MI_WLAN_Encrypt_e;

```

➤ Member

Member	Description
E_MI_WLAN_ENCRYPT_NONE	WLAN module encrypt type none
E_MI_WLAN_ENCRYPT_UNKNOWN	WLAN module encrypt type unknown
E_MI_WLAN_ENCRYPT_WEP	WLAN module encrypt type WEP
E_MI_WLAN_ENCRYPT_TKIP	WLAN module encrypt type TKIP
E_MI_WLAN_ENCRYPT_AES	WLAN module encrypt type AES
E_MI_WLAN_ENCRYPT_MAX	WLAN module max

➤ Note

N/A

➤ Related Data Type and Interface

N/A

2.1.4 MI_WLAN_NetworkType_e

➤ Description

WLAN device network type

➤ Definition

```

typedef enum
{
    /// WLAN network infrastructure type
    E_MI_WLAN_NETWORKTYPE_INFRA,
    /// WLAN network AP type
    E_MI_WLAN_NETWORKTYPE_AP,
    /// WLAN network AdHoc type
    E_MI_WLAN_NETWORKTYPE_ADHOC,
    /// WLAN network Monitor type
    E_MI_WLAN_NETWORKTYPE_MONITOR,
    /// WLAN network mode master
    E_MI_WLAN_NETWORKTYPE_MASTER,
    /// WLAN network mode slave
    E_MI_WLAN_NETWORKTYPE_SLAVE,
    /// WLAN param max
    E_MI_WLAN_NETWORKTYPE_MAX
} MI_WLAN_NetworkType_e;

```

➤ Member

Member	Description
E_MI_WLAN_NETWORKTYPE_INFRA	WLAN network infrastructure type(sta)
E_MI_WLAN_NETWORKTYPE_AP	WLAN network AP type
E_MI_WLAN_NETWORKTYPE_ADHOC	WLAN network AdHoc type
E_MI_WLAN_NETWORKTYPE_MONITOR	WLAN network Monitor type
E_MI_WLAN_NETWORKTYPE_MASTER	WLAN network mode master
E_MI_WLAN_NETWORKTYPE_SLAVE	WLAN network mode slave
E_MI_WLAN_NETWORKTYPE_MAX	WLAN param max

➤ Note

Types supported include E_MI_WLAN_NETWORKTYPE_INFRA and E_MI_WLAN_NETWORKTYPE_AP.

➤ Related Data Type and Interface
N/A

2.1.5 MI_WLAN_Authentication_Suite_e

➤ Description

Wi-Fi authentication suite

➤ Definition

```
typedef enum
{
    /// Authentication Suite PSK
    E_MI_WLAN_AUTH_SUITE_PSK,
    /// Authentication Suite unknown
    E_MI_WLAN_AUTH_SUITE_UNKNOWN,
    E_MI_WLAN_AUTH_SUITE_MAX
} MI_WLAN_Authentication_Suite_e;
```

➤ Member

Member	Description
E_MI_WLAN_AUTH_SUITE_PSK	WLAN Authentication Suite PSK
E_MI_WLAN_AUTH_SUITE_UNKNOWN	WLAN Authentication Suite unknown
E_MI_WLAN_AUTH_SUITE_MAX	WLAN Param Max

➤ Note

N/A

➤ Related Data Type and Interface
N/A

2.1.6 MI_WLAN_WPAStatus_e

➤ Description

Wi-Fi connection status

➤ Definition

```
typedef enum
{
    WPA_DISCONNECTED,
    WPA_INTERFACE_DISABLED,
    WPA_INACTIVE,
    WPA_SCANNING,
    WPA_AUTHENTICATING,
    WPA_ASSOCIATING,
    WPA_ASSOCIATED,
    WPA_4WAY_HANDSHAKE,
    WPA_GROUP_HANDSHAKE,
    WPA_COMPLETED
} MI_WLAN_WPAStatus_e;
```

➤ Member

Member	Description
WPA_DISCONNECTED	WPA Disconnected status
WPA_INTERFACE_DISABLED	WPA Interface disable status
WPA_INACTIVE	WPA Inactive status
WPA_SCANNING	WPA Scanning status
WPA_AUTHENTICATING	WPA Authenticating status
WPA_ASSOCIATING	WPA Associating status
WPA_ASSOCIATED	WPA Associated status
WPA_4WAY_HANDSHAKE	WPA 4 way handshake
WPA_GROUP_HANDSHAKE	WPA group handshake
WPA_COMPLETED	WPA completed

➤ Note

N/A

➤ Related Data Type and Interface

N/A

2.1.7 MI_WLAN_InitParams_t

➤ Description

WLAN device initialization parameter

➤ Definition

```
typedef struct MI_WLAN_InitParams_s
{
    /// json description file of Wi-Fi dongle
    MI_U8 au8JsonConfFilePath[MI_WLAN_MAX_FOLDERPATH_LEN];
```



```

    /// reserved
    MI_U64 u64Reserved;
} MI_WLAN_InitParams_t;

```

➤ Member

Member	Description
au8JsonConfFilePath	Absolute path for the json configuration file required of WLAN device initialization
u64Reserved	Reserved and not used currently.

➤ Note

The initialization of WLAN device relies on a json configuration file called wlan.json and located at /config/Wi-Fi/wlan.json.
Note that currently this parameter does not accept Null value.

➤ Related Data Type and Interface

N/A

2.1.8 MI_WLAN_OpenParams_t

➤ Description

WLAN device enable parameter

➤ Definition

```

typedef struct MI_WLAN_OpenParam_s
{
    // WLAN network type
    MI_WLAN_NetworkType_e eNetworkType;
    // reserved
    MI_BOOL bReserved;
} MI_WLAN_OpenParams_t;

```

➤ Member

Member	Description
MI_WLAN_NetworkType_e	WLAN device network type
bReserved	Reserved and not used currently.

➤ Note

N/A

➤ Related Data Type and Interface

[MI_WLAN_NetworkType_e](#)

2.1.9 MI_WLAN_ConnectParam_t

➤ Description

WLAN device connection parameter setting

➤ Definition

```
typedef struct
{
    // WLAN security mode
    MI_WLAN_Security_e eSecurity;
    // WLAN SSID
    MI_U8 au8SSID[MI_WLAN_MAX_SSID_LEN];
    // WLAN password
    MI_U8 au8Password[MI_WLAN_MAX_PASSWD_LEN];
    // WLAN connect overtime
    MI_U32 OverTimeMs;
} MI_WLAN_ConnectParam_t;
```

➤ Member

Member	Description
MI_WLAN_Security_e	Under infra mode, this refers to Wi-Fi hotspot security type. Under ap mode, this refers to the security type applied when connecting Wi-Fi network.
au8SSID	Wi-Fi hotspot name
au8Password	Wi-Fi hotspot password
OverTimeMs	Specify the Wi-Fi connection timeout time

➤ Note

WLAN module security key WPA and WLAN module security key WPA2 are suggested.

➤ Related Data Type and Interface

[MI_WLAN_NetworkType_e](#)

[MI_WLAN_Security_e](#)

2.1.10 MI_WLAN_Quality_t

➤ Description

Wi-Fi hotspot signal quality

➤ Definition

```
typedef struct
{
    MI_U8 curLVL;
    MI_U8 maxLVL;
    MI_S8 signalSTR;
} MI_WLAN_Quality_t;
```

➤ Member

Member	Description
curLVL	Current Wi-Fi hotspot signal level
maxLVL	Global Wi-Fi hotspot maximum signal level
signalSTR	Current Wi-Fi hotspot signal strength (mdb)

➤ Note

N/A

- Related Data Type and Interface
N/A

2.1.11 MI_WLAN_Cipher_t

- Description
Wi-Fi connection cipher setting

- Definition

```
typedef struct
{
    // WLAN Security mode
    MI_WLAN_Security_e eSecurity;
    // WLAN Encryption type
    MI_WLAN_Encrypt_e eGroupCipher;
    // WLAN Encryption type
    MI_WLAN_Encrypt_e ePairCipher;
    // WLAN authentication suite
    MI_WLAN_Authentication_Suite_e eAuthSuite;
} MI_WLAN_Cipher_t;
```

- Member

Member	Description
eSecurity	Wi-Fi hotspot security type
eGroupCipher	Wi-Fi hotspot group cipher type
ePairCipher	Wi-Fi hotspot pair cipher type
eAuthSuite	Wi-Fi connection authentication suite

- Note
N/A

- Related Data Type and Interface
[MI_WLAN_Security_e](#)
[MI_WLAN_Encrypt_e](#)
[MI_WLAN_Authentication_Suite_e](#)

2.1.12 MI_WLAN_APIInfo_t

- Description
Wi-Fi hotspot information
- Definition

```
typedef struct MI_WLAN_APIInfo_s
{
    // WLAN CELL ID
    MI_U16 u16CellId;
    // WLAN Frequency GHz
    MI_FLOAT fFrequency;
    // WLAN Bitrate Mb/s
```

```

MI_FLOAT fBitRate;
// WLAN Quality
MI_WLAN_Quality_t stQuality;
// WLAN Encryption key on/off
MI_BOOL bEncryptKey;
// WLAN SSID
MI_U8 au8SSID[MI_WLAN_MAX_SSID_LEN];
// WLAN Channel
MI_U8 u8Channel;
// WLAN MAC
MI_U8 au8Mac[MI_WLAN_MAX_MAC_LEN];
// WLAN Encryption type
MI_WLAN_Encrypt_e eEncrypt;
// WLAN AP type (Infrastructure / Ad-Hoc)
MI_WLAN_NetworkType_e eMode;
// WLAN cipher kit
MI_WLAN_Cipher_t stCipher[2];
} MI_WLAN_APIInfo_t;

```

➤ Member

Member	Description
u16CellId	Wi-Fi hotspot ID
fFrequency	Wi-Fi hotspot operation frequency
fBitRate	Wi-Fi hotspot bitrate
stQuality	Wi-Fi hotspot quality
bEncryptKey	Wi-Fi hotspot password required or not
au8SSID	Wi-Fi hotspot ID
u8Channel	Wi-Fi hotspot channel
au8Mac	Wi-Fi hotspot Mac address
eEncrypt	Wi-Fi hotspot encryption
eMode	Wi-Fi hotspot work mode
stCipher	Wi-Fi hotspot cypher suite

➤ Note

N/A

➤ Related Data Type and Interface

[MI_WLAN_Quality_t](#)

[MI_WLAN_Encrypt_e](#)

[MI_WLAN_NetworkType_e](#)

[MI_WLAN_Cipher_t](#)

2.1.13 MI_WLAN_ScanParam_t

➤ Description

Wi-Fi hotspot scan parameter setting

➤ Definition

```

typedef struct MI_WLAN_ScanParam_s
{

```

```
// Wlan set block mode
MI_BOOL bBlock; // reserved and unused at present
} MI_WLAN_ScanParam_t;
```

➤ Member

Member	Description
bBlock	Indicate whether or not to execute this function on block mode. This parameter is not used currently. The block setting can be configured in the json configuration file.

➤ Note

N/A

➤ Related Data Type and Interface

2.1.14 MI_WLAN_ScanResult_t

➤ Description

Wi-Fi hotspot scan result

➤ Definition

```
typedef struct MI_WLAN_ScanResult_s
{
    // Wlan AP number
    MI\_WLAN\_APIInfo\_t stAPIInfo[MI_WLAN_MAX_APIINFO_NUM];
    MI_U8 u8APNumber;
} MI_WLAN_ScanResult_t;
```

➤ Member

Member	Description
stAPIInfo	Reserved and not used currently Details click on MI_WLAN_APIInfo_t
u8APNumber	AP Number

➤ Note

N/A

➤ Related Data Type and Interface

[MI_WLAN_APIInfo_t](#)

2.1.15 MI_WLAN_Status_sta_t

➤ Description

Wi-Fi connection status

➤ Definition

```
typedef struct MI_WLAN_Status_sta_s
{
```

```

MI_U8 bssid[MI_WLAN_BSSID_LEN];
MI_U32 freq;
MI_U8 ssid[MI_WLAN_MAX_SSID_LEN];
MI_U16 id;
MI_WLAN_NetworkType_e mode;
MI_WLAN_Cipher_t stCipher;
MI_U8 key_mgmt[12];
MI_WLAN_WPAStatus_e state;
MI_U8 address[MI_WLAN_BSSID_LEN];
MI_U8 ip_address[16];
MI_U32 channel;
MI_U32 RSSI;
MI_U8 Bandwidth[8];
} MI_WLAN_Status_sta_t;

```

➤ Member

Member	Description
bssid	Wi-Fi hotspot ID in hexadecimal representation
freq	Connected Wi-Fi hotspot frequency
ssid	Wi-Fi hotspot ID
id	Wi-Fi hotspot ID
mode	Wi-Fi hotspot work mode
stCipher	Wi-Fi hotspot cipher suite
key_mgmt	Wi-Fi hotspot private key management type
state	Wi-Fi hotspot current connection status
address	Wi-Fi hotspot host IP address
channel	Currently connected Wi-Fi channel
RSSI	Currently connected Wi-Fi RSSI level
Bandwidth	Currently connected Wi-Fi bandwidth

➤ Note

N/A

➤ Related Data Type and Interface

[MI_WLAN_NetworkType_e](#)

[MI_WLAN_Cipher_t](#)

[MI_WLAN_WPAStatus_e](#)

2.1.16 MI_WLAN_Status_host_t

➤ Description

Host Wi-Fi connection status under AP mode

➤ Definition

```

typedef struct MI_WLAN_Status_host_s
{
    MI_U8 hostname[MI_WLAN_MAX_HOST_NAME_LEN];
    MI_U8 ipaddr[16];
}

```

```

    MI_U8 macaddr[18];
    MI_U64 connectedtime;

} MI_WLAN_Status_host_t;

```

➤ Member

Member	Description
hostname	Connected host name
ipaddr	Connected host IP address
macaddr	Connected host Mac address
connectedtime	Connected host connected time

➤ Note

N/A

➤ Related Data Type and Interface

N/A

2.1.17 MI_WLAN_Status_t

➤ Description

Connection status of all connected Wi-Fi networks

➤ Definition

```

typedef union MI_WLAN_Status_s
{
    MI\_WLAN\_Status\_sta\_t stStaStatus;
    MI\_WLAN\_Status\_ap\_t stApStatus;
} MI_WLAN_Status_t;

```

➤ Member

Member	Description
stStaStatus	Currently connected Wi-Fi hotspot status
stApStatus	Currently connected host status

➤ Note

N/A

➤ Related Data Type and Interface

[MI_WLAN_Status_sta_t](#)
[MI_WLAN_Status_ap_t](#)

2.1.18 MI_WLAN_Status_ap_t

➤ Description

Define all connected host information

➤ Definition

```
typedef struct MI_WLAN_Status_ap_s
{
    MI\_WLAN\_Status\_host\_t astHosts[MI_WLAN_MAX_HOSTS];
    MI_U16 u16HostNum;
} MI_WLAN_Status_ap_t;
```

➤ Member

Member	Description
astHosts	Dev hosts AP connected
u16HostNum	Host Ap Num

➤ Note

N/A

➤ Related Data Type and Interface

[MI_WLAN_Status_sta_t](#)

[MI_WLAN_Status_host_t](#)

3. ERROR CODE

The table below lists the WLAN API error codes:

Table 1: WLAN API Error Code

Macro Definition	Description
MI_WLAN_ERR_FAIL	Error unidentified
MI_WLAN_ERR_INVALID_DEVID	Invalid DEV ID
MI_WLAN_ERR_ILLEGAL_PARAM	Illegal input parameter
MI_WLAN_ERR_NOT_SUPPORT	Input parameter type not supported
MI_WLAN_ERR_MOD_INITED	Device already initialized
MI_WLAN_ERR_MOD_NOT_INIT	Device not yet initialized
MI_WLAN_ERR_NOT_CONFIG	Device not configured
MI_WLAN_ERR_INVAL_HANDLE	Invalid handle
MI_WLAN_ERR_NULL_PTR	Null input parameter
MI_WLAN_ERR_INITED	Module already initialized

4. APPENDIX A

4.1. wlan.json

```
{
  /*currently selected Wi-Fi device, multiple Wi-Fi devices are supported by this configuration file.
  **Since ssw01b is taken as an example in the following configuration file, MI_WLAN will parse the information under the ssw01b tag.
  */
  "selfie": {
    "Wi-Fi": "ssw01b"
  },
  /*Wi-Fi device tag*/
  "ssw01b": {
    /*Wi-Fi device self-description*/
    "selfie": {
      /*Wi-Fi device firmware version*/
      "version": "0.0.0",
      /*Wi-Fi device name*/
      "name": "ssw01b40m",
      /*Wi-Fi device initialization script path, which will be called by MI_WLAN_Init*/
      "scriptPath": "/config/Wi-Fi",
      /*The default script parser for all "script"tags of this Wi-Fi device
      **The script does not necessarily have to be shell; it can be Python perl, etc.
      */
      "parser": "sh"
    },
    /*Wi-Fi initialization tag
    ** Parsed and executed in the MI_WLAN_Init function
    */
    "init": {
      /*Linux environment variables to be configured for operation of this Wi-Fi device
      **The name and value of the environment variables must correspond
      **Number of env variables is not restricted
      */
      "env": {
        "name": ["LD_LIBRARY_PATH","PATH"],
        "value": ["/config/Wi-Fi","/config/Wi-Fi"]
      },
      /*scripts to be parsed for initialization of this Wi-Fi device
      **@parser The bin file for parsing this script, optional tag
      **@name Name of the script file
      **@option Parameter of the script, number not restricted, optional tag
```

```

    */
    "script": {
        "parser": "source",
        "name": "ssw01bInit.sh",
        "option": ["dummy1", "dummy2"]
    }
},
/*Wi-Fi device deinitialization tag*/
"deinit": {
    /*Scripts which should be parsed for deinitialization of this Wi-Fi device
    **@parser optional tag, not enabled
    **@name script file
    **@option optional tag, not enabled
    */
    "script": {
        "name": "ssw01bDeInit.sh"
    }
},
/*Wi-Fi device universal info configuration tag
**This tag provides some basic settings required of Wi-Fi services
**User can customize the settings according to his own Wi-Fi operation requirements, by
**configuring the contents under the tag
*/
"universal": {
    /*tag to be configured under host mode*/
    "infra": {
        /*name of interface to Wi-Fi hotspot provided by Wi-Fi device
        **name of interface provided by different Wi-Fi device can be different
        */
        "iface": "wlan0",
        /*directory of control interface provided by system to Wi-Fi device
        **must be created before initialization, default created by the script contained in the initialization tag
        **path must be synchronized at user end
        */
        "ctrl_interface": "/config/Wi-Fi/run/wpa_supplicant"
    },
    /*tag to be configured under hotspot mode*/
    "ap": {
        /*name of interface to Wi-Fi hotspot provided by Wi-Fi device
        ** name of interface provided by different Wi-Fi device can be different
        */
        "iface": "p2p0",
        /*static IP address of the hotspot*/
        "ipaddr": "192.168.1.100",
        /*netmask of the hotspot*/
        "netmask": "255.255.255.0",

```

```

/* directory of control interface provided by system to Wi-Fi device
** must be created before initialization, default created by the script contained in the initialization tag
** path must be synchronized at user end
*/
"ctrl_interface": "/var/run/hostapd"

},
/*udhcpd path of the script to be configured for getting network connection information, provided by system*/
"dhcp_script": "/etc/init.d/udhcpd.script",
/*wpa_supplicant configuration file path, which should be created by user*/
"wpa_supplicant_conf": "/config/Wi-Fi/wpa_supplicant.conf",
/*hostapd configuration file path, which should be created by user */
"hostapd_conf": "/config/Wi-Fi/hostapd.conf",
/*dnsmasq configuration file path, which should be created by user */
"dnsmasq_conf": "/config/Wi-Fi/dnsmasq.conf",
/*dhcp lease file, the directory of which should be created by user*/
"dhcp-leasefile": "/var/lib/misc/dnsmasq.leases"
},
/*
**individual tag defines all actions supported by MI_WLAN
**action subtag, which defines the following:
**@scan scan operation
**@open opening WLAN device
**@close closing WLAN device
**@connect connecting Wi-Fi service
**@disconnect disconnecting Wi-Fi service
**@status getting current Wi-Fi connection status
**behavior will be different depending on the operation mode of each action
**pattern of action is defined as follows:
**{
    "binary":["bin1",...."binx"],
    "option0":["opt1",...."optX"],
    .....
    .....
    .....
    "option#N":["opt1",...."optX"]
**}
**binary bin file and option array have a one-to-one relationship. The tag of option contains a specific numbered suffix, starting from 0.
**option contents, which support content reference to universal tag. The syntax '$TAGNAME' supports level-dependent content reference
**'$TAGNAME:$SUB_TAGNAME'
}
*/
"individual": {
    "action": {
        "scan": {
            "binary": ["iwlist"],

```

```
"option0": ["$infra:$iface", "scanning"]
},
"open": {
  "deviceup": {
    "ap": {
      "binary": ["ifconfig"],
      "option0": ["$ap:$iface", "up"]
    },
    "infra": {
      "binary": ["ifconfig"],
      "option0": ["$infra:$iface", "up"]
    }
  },
  "serviceup": {
    "ap": {
      "binary": ["ifconfig", "hostapd"],
      "option0": ["$ap:$iface", "$ap:$ipaddr", "netmask", "$ap:$netmask"],
      "option1": ["-B", "$hostapd_conf"]
    },
    "infra": {
      "binary": ["wpa_supplicant"],
      "option0": ["-i", "$infra:$iface", "-Dnl80211", "-c", "$wpa_supplicant_conf", "-d", "&"]
    }
  }
},
"close": {
  "devicedown": {
    "ap": {
      "binary": ["ifconfig"],
      "option0": ["$ap:$iface", "down"]
    },
    "infra": {
      "binary": ["ifconfig"],
      "option0": ["$infra:$iface", "down"]
    }
  },
  "servicedown": {
    "ap": {
      "script": {
        "parser": "sh",
        "name": "atbmClose.sh",
        "option": ["ap"]
      }
    },
    "infra": {
      "script": {
```

```

        "name": "atbmClose.sh",
        "option": ["infra"]
    }
}
},
"connect": {
    "ap": {
        "binary": ["ifconfig", "dnsmasq"],
        "option0": ["$ap:$iface", "up"],
        "option1": ["-i", "$ap:$iface", "--no-daemon", "-C", "$dnsmasq_conf", "&"]
    },
    "infra": {
        "add": {
            "binary": ["wpa_cli"],
            "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "add_network"]
        },
        "setup": {
            "ssid": {
                "binary": ["wpa_cli"],
                "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "set_network", "%d", "ssid", "\\\\" "%s\\\\""]
            },
            "password": {
                {
                    "keyon": {
                        "wpa": {
                            "binary": ["wpa_cli"],
                            "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "set_network", "%d", "psk", "\\\\" "%s\\\\""]
                        },
                        "wep": {
                            "binary": ["wpa_cli"],
                            "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "set_network", "%d", "wep_key0", "\\\\" "%s\\\\""]
                        }
                    },
                    "keyoff": {
                        "binary": ["wpa_cli"],
                        "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "set_network", "%d", "key_mgmt", "NONE"]
                    }
                }
            }
        },
        "enable": {
            "binary": ["wpa_cli", "wpa_cli"],
            "option0": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "select_network", "%d"],
            "option1": ["-i", "$infra:$iface", "-p", "$infra:$ctrl_interface", "enable_network", "%d"]
        }
    },
}

```

```
"dhcpc": {
  "binary": ["udhcpc"],
  "option0": ["-i","$infra:$iface","-s","$dhcp_script","-a","-q","-b","-t","20","-T","1"]
},
"save": {
  "binary": ["wpa_cli"],
  "option0": ["-i","$infra:$iface","-p","$infra:$ctrl_interface","save_config"]
}
},
"disconnect": {
  "ap": {
    "binary": ["ifconfig"],
    "option0": ["$ap:$iface","down"]
  },
  "infra": {
    "binary": ["wpa_cli"],
    "option0": ["-i","$infra:$iface","-p","$infra:$ctrl_interface","disable_network","%d"]
  }
},
"status": {
  "ap": {
    "all_sta":{
      "binary": ["hostapd_cli"],
      "option0": ["-i","$ap:$iface","-p","$ap:$ctrl_interface","all_sta"]
    }
  },
  "infra": {
    "binary": ["wpa_cli"],
    "option0": ["-i","$infra:$iface","-p","$infra:$ctrl_interface","status"]
  }
}
}
}
}
```